



Unidad 1

Introducción a los lenguajes de marcas

Módulo: Llenguatges de Marques i Sistemes de Gestió d'Informació (LMSGI)

Ciclo formativo: 1r ASIX (Administración de Sistemas Informáticos en Red) · 1r DAW (Desarrollo de Aplicaciones Web)

Curso: 2026-2027

Docente: Noel Marco Biendicho

Centro: IES Sant Vicent Ferrer — Algemesí

Índice

1. ¿Qué es un lenguaje de marcas? (CA 1a)
2. Ventajas de los lenguajes de marcas (CA 1b)
3. Origen y clasificación de los lenguajes de marcas (CA 1c, 1d)
4. Características de los lenguajes de marcas
5. Ámbitos de aplicación y XML como lenguaje de propósito general (CA 1d, 1e)
6. XML: estructura y sintaxis (CA 1f, 1g)
7. Documentos XML bien formados (CA 1h)
8. Espacios de nombres en XML (CA 1i)
9. Resumen de la unidad
10. Para saber más

RA cubierto: RA1 — *Reconoce las características de los lenguajes de marcas analizando e interpretando fragmentos de código* (CA 1a-1i)

💡 **Nota de estudio:** en esta unidad hay poca "teoría para memorizar" y mucha idea que necesitas entender para poder programar después. Si un concepto no queda claro a la primera, puedes repasarlo por tu cuenta — lo importante es que sepas *reconocer* un lenguaje de marcas y *escribir* un XML correcto al final de la unidad.

1. ¿Qué es un lenguaje de marcas? (CA 1a)

Un **lenguaje de marcas** (o *markup language*) es un sistema para **dar formato o estructura a un texto**, insertando dentro del propio texto unas señales especiales llamadas **marcas** (o *etiquetas*), sin necesidad de un lenguaje de programación.

- El término viene del inglés *marking up*: la técnica de marcar manuscritos con lápiz de color para anotar, por ejemplo, la tipografía que debía usar el impresor.
- Una marca es una señal colocada en el texto, normalmente encerrada entre los símbolos `<` y `>`, y suele aparecer por parejas (apertura y cierre).
- Un programa (el **user-agent**) interpreta esas marcas para saber qué hacer con el texto que envuelven. La marca en sí no cambia, pero **cada user-agent decide qué hacer con ella** — por eso el mismo documento marcado puede dar resultados muy distintos según quién lo procese:
 - **Navegador** (Chrome, Firefox...): interpreta `<h1>`, `<table>`, `<img>`... y las convierte en texto renderizado, tablas visuales o imágenes en pantalla.
 - **Editor de código** (VS Code, Sublime...): interpreta esas mismas marcas para resaltar sintaxis en colores, plegar bloques, o mostrar una vista previa — como la que usas para leer este propio documento en Markdown.
 - **Lector de pantalla** (herramienta de accesibilidad): interpreta marcas semánticas como `<nav>`, `<button>` o `<h2>` no para pintarlas, sino para anunciarlas por voz a una persona con discapacidad visual ("encabezado de nivel 2: Ventajas de los lenguajes de marcas").
 - **Sintetizador de voz**: interpreta marcas SSML como `<break time="500ms"/>` (pausa) o `<emphasis>` (énfasis) para dar la entonación correcta al convertir texto en audio.
 - **Procesador de texto** (Word, LibreOffice): al abrir un `.docx`, interpreta por debajo las marcas XML internas del archivo para mostrar negritas, estilos o tablas, aunque tú nunca veas ese XML directamente.

Esto es justo la base de la **independencia del dispositivo** que veremos en el punto 2 (Ventajas): el mismo documento marcado puede acabar como página web, audio o documento impreso, según qué user-agent lo interprete.

```
<h1>Texto grande</h1>
<h3>Texto pequeño</h3>
```

💡 **Los encabezados van de <h1> a <h6>** — seis niveles, sin más allá (no existe <h7>) y sin nada "por encima" de <h1>. No indican un tamaño de letra: indican **nivel de importancia/jerarquía** dentro del documento (como los niveles de un índice). Que <h1> se vea más grande que <h3> es solo el estilo que aplica el navegador **por defecto**; con CSS se puede cambiar el tamaño visual sin tocar esa jerarquía. El texto "normal" no lleva ninguna etiqueta de encabezado — se escribe dentro de un párrafo (<p>).

⚠️ **Importante:** no hay que confundir un lenguaje de marcas (LM) con un **lenguaje de programación** (LP). Un LM no tiene variables, bucles ni funciones — solo describe formato o estructura. Cuando un LM se combina con un LP (por ejemplo HTML + JavaScript), decimos que el documento se ha "*programado*", pero el HTML en sí mismo no es programación.

🔧 Ejercicio 1 — Tu primer documento marcado

1. Crea un archivo de texto llamado `textos.txt`.
2. Escribe dentro:

```
<h1>Texto grande</h1>
<h3>Texto pequeño</h3>
```

3. Ábrelo con un navegador. ¿Qué ves?
4. Renómbalo a `textos.html` y vuelve a abrirlo. ¿Qué cambia?
5. Abre ese mismo `textos.html` con un editor de código, mostrando el archivo en bruto (sin vista previa). Ya tienes dos *user-agents* distintos interpretando el mismo documento marcado: el navegador y el editor. Explica con tus palabras qué hace cada uno con las mismas marcas, y qué pasaría si en vez de un navegador lo "abriera" un lector de pantalla.

2. Ventajas de los lenguajes de marcas (CA 1b)

- **Texto plano:** se pueden leer, escribir y editar con cualquier editor de texto, sin software específico ni licencias. Si abres un `.html`, `.xml` o `.md` con el Bloc de notas ves el código directamente; un binario (una imagen, un ejecutable, un `.doc` antiguo) se ve ilegible sin el programa que lo generó.
 - **Control de versiones:** al ser texto, herramientas como Git pueden comparar dos versiones línea a línea y mostrar exactamente qué ha cambiado:

```
- <button class="btn-primary">Enviar</button>
+ <button class="btn-success">Enviar</button>
```

Con un binario, Git solo puede decir "esto es distinto", no "qué ha cambiado" — y si dos personas lo editan a la vez, no hay forma de combinar ambos cambios automáticamente.

- **Matiz:** hoy en día formatos como `.docx` o `.xlsx` son, por dentro, XML comprimido en un ZIP — "texto plano" en su núcleo —, pero eso no los hace editables con un editor de texto normal, porque el usuario nunca toca ese XML directamente: lo gestiona el programa.

- **Independencia del dispositivo:** el mismo documento puede interpretarse de formas distintas según el dispositivo final (pantalla, impresora, móvil) — el LM no depende de dónde se muestre, porque solo describe **qué es** el contenido, no cómo debe verse exactamente en cada caso.
 - Ejemplo real: una misma página web se ve en una columna en el móvil y en varias en el ordenador (diseño responsive), y al imprimirla muchos sitios ocultan el menú de navegación con una hoja de estilos de impresión — el HTML no cambia, solo la interpretación que hace cada salida.
 - Esta independencia es la razón de ser de **separar contenido y presentación**. Contenido neutro (HTML):

```
<h1>Título</h1>
```

Presentación aparte (CSS), con reglas que pueden variar según el contexto:

```
h1 { color: blue; font-size: 3em; }  
@media print { h1 { color: black; font-size: 1.5em; } }
```

Si el formato estuviera mezclado dentro del contenido (`<font size="7" color="blue">Título</font>`), haría falta una copia distinta del documento para cada dispositivo.

- Conecta directamente con el punto 1: cada user-agent (navegador, lector de pantalla, sintetizador de voz...) es, en el fondo, un "dispositivo final" distinto interpretando el mismo documento marcado.
- **Facilidad de intercambio:** al ser texto con una sintaxis pública y documentada, cualquier sistema puede leerlo y escribirlo sin necesitar el programa original.
 - Antes de que se generalizaran los formatos de texto, cada aplicación guardaba sus datos en un formato binario propio, lo que hacía casi imposible pasar un documento de una aplicación a otra distinta — emisor y receptor no "hablaban el mismo idioma".
 - Ejemplos actuales: un lector RSS de cualquier fabricante puede leer el feed de cualquier blog (todos son XML); Word, LibreOffice y Google Docs abren el mismo `.docx`; dos sistemas hechos en lenguajes de programación distintos pueden intercambiar datos en JSON o XML sin necesitar un "traductor" a medida.
- **Flexibilidad:** un LM se puede combinar con otros lenguajes fácilmente, sin dejar de ser él mismo.
 - Con lenguajes de programación: HTML puede incorporar JavaScript (`<script>`) o PHP (`<?php ?>`) para añadir comportamiento dinámico — el LM aporta la estructura, el lenguaje de programación el comportamiento.
 - Con otros lenguajes de marcas: un `<svg>` (gráfico vectorial) puede insertarse directamente dentro de una página HTML, o datos estructurados en JSON pueden incluirse en una etiqueta `<script type="application/ld+json">` para que los buscadores los lean.

Ejercicio 2 — Texto plano frente a binario

1. Crea un archivo `nota.html` con un `<h1>` y un `<p>` cualquiera.
2. Ábrelo con el Bloc de notas (o cualquier editor de texto plano) y comprueba que ves el código tal cual.
3. Coge cualquier archivo binario que tengas a mano (una imagen `.jpg`, un `.docx`, un ejecutable) y ábrelo también con el Bloc de notas. ¿Qué ves esta vez?
4. Modifica una palabra de `nota.html`, guarda, y compara con la versión anterior usando `git diff` (o cualquier comparador de texto). ¿Podrías hacer lo mismo con el archivo binario del paso 3? Relaciona tu respuesta con la ventaja de "texto plano" de este punto.

3. Origen y clasificación de los lenguajes de marcas (CA 1c, 1d)

3.1. De dónde viene todo esto

En los inicios de la informática, cada aplicación usaba sus propias marcas, lo que hacía imposible intercambiar documentos entre plataformas distintas.

- **GML** (*Generalized Markup Language*), desarrollado en IBM en 1969-1970 por Charles Goldfarb, Edward Mosher y Raymond Lorie (sus iniciales dan nombre al lenguaje), fue el primero en independizar el documento del dispositivo, usando marcas genéricas.
- **SGML** (*Standard GML*), estandarización de GML por ISO en 1986 liderada por Goldfarb, es un **metalenguaje**: no tiene etiquetas propias, sino que define las reglas para crear otros lenguajes de marcas. De él nacen tanto XML como HTML, aunque de forma distinta:
 - **XML** (1996-98) es un **subconjunto simplificado de SGML**: se creó explícitamente para conservar lo esencial pero con una sintaxis mucho más sencilla de procesar — SGML se consideraba demasiado complejo.
 - **HTML** (1991-93) es, en cambio, una **aplicación de SGML**: Tim Berners-Lee lo definió usando las reglas de SGML (con su propio DTD), pero nunca ha sido tan estricto como su modelo — los navegadores siempre lo han interpretado de forma permisiva, "perdonando" errores que SGML (y XML) rechazarían directamente.

 **Esto es historia, no una herramienta que se use hoy**: prácticamente nadie escribe documentos SGML directamente en la actualidad (salvo casos muy residuales en algunos sectores editoriales). Se estudia porque explica *por qué* HTML y XML son como son, no porque vayas a usarlo.

Un documento SGML se compone de tres partes:

- una **declaración** (indica que el documento es SGML),
- un **DTD** (*Document Type Definition*: define la sintaxis concreta del lenguaje creado — qué etiquetas existen y cómo se pueden combinar),
- una **instancia** (los datos reales).

Ejemplo — definición de un vocabulario para módulos de un ciclo formativo:

```
Vocabulario: daw, modulo, titulo, contenido, unidad
Reglas: daw contiene varios modulos; un modulo tiene un titulo y un contenido;
        contenido tiene varias unidades; las unidades son texto simple.
```

```
<daw>
  <modulo>
    <titulo>Lenguaje de Marcas</titulo>
    <contenido>
      <unidad>Introduccion</unidad>
      <unidad>HTML</unidad>
      <unidad>CSS</unidad>
    </contenido>
  </modulo>
</daw>
```

3.2. Otros lenguajes de marcas relevantes

Lenguaje	Año	Para qué sirve	¿Sigue vigente?
TeX / LaTeX	años 70 (Donald Knuth)	Producir documentos científicos de gran calidad tipográfica, con la misma apariencia en cualquier equipo	✓ Sí — sigue siendo el estándar en publicaciones científicas y matemáticas (uso de nicho, no generalista)
RTF (<i>Rich Text Format</i>)	1987 (Microsoft)	Documentos de texto con anotaciones de formato, intercambiables entre procesadores de texto	⚠ Uso residual — quedó en segundo plano frente a formatos como .docx
PostScript	1976 (Warnock, luego Adobe)	Lenguaje de descripción de páginas para impresión	⚠ Uso residual — sustituido en gran parte por PDF
HTML	1992 (Tim Berners-Lee)	Crear páginas web con hipertexto	✓ Totalmente vigente (ver nota más abajo)
XML	1998	Metalenguaje para intercambio de datos estructurados	✓ Muy vigente (ver nota más abajo)
JSON	2001	Notación de datos ligera, basada en JavaScript	✓ Vigente — hoy domina el intercambio de datos en APIs web, compitiendo con XML
Markdown	2004 (John Gruber)	Marcado ligero orientado a presentación, pensado para convertirse fácilmente a HTML	✓ Muy vigente — documentación técnica, GitHub, notas... este mismo documento está escrito en Markdown

Ejemplo de LaTeX:

```
\documentclass[12pt]{article}
\begin{document}
Este es el texto ejemplo de \LaTeX{}
```

Con datos en `\emph{cursiva}` o `\textbf{negrita}`.
`\end{document}`

 **Sigue vigente, y con herramientas modernas:** hoy apenas se escribe LaTeX en un editor de texto suelto — se usan plataformas como Overleaf, que permiten escribir y compilar el documento en el navegador y colaborar en tiempo real, como un Google Docs para LaTeX. Su punto fuerte real es el manejo de fórmulas matemáticas complejas y una tipografía muy cuidada, algo que Word gestiona peor cuanto más compleja es la fórmula.

Ejemplo de JSON:

```
{
  "nombre": "Jorge",
  "apellido": "Sánchez",
  "telefonos": [
    { "tipo": "fijo", "numero": "999999999" },
    { "tipo": "movil", "numero": "666666666" }
  ]
}
```

 **El formato de facto para APIs web:** aunque su nombre venga de *JavaScript Object Notation*, hoy lo usa prácticamente cualquier lenguaje de programación — no está atado a JavaScript. Su estructura es más simple que la de XML (solo objetos {}, listas [], texto, números, booleanos y `null`), lo que lo hace más ligero de leer y escribir — a cambio, a diferencia de XML, no admite comentarios dentro del propio archivo.

 **Actualidad — HTML ya no tiene "versiones":** desde 2019, el **W3C cedió la autoridad** sobre el estándar de HTML al **WHATWG**, que lo mantiene como un *"Living Standard"* (estándar vivo): un único documento que se actualiza continuamente, sin congelarse nunca en un número de versión cerrado. Hablar de "HTML5" hoy es más una etiqueta de marketing que una versión formal — el estándar de referencia actual vive en html.spec.whatwg.org.

 **Actualidad — XML no ha muerto, ha cambiado de terreno:** JSON le ha ganado la partida a XML en las **APIs web** (más ligero, más fácil de leer). Pero XML sigue siendo la base de: los formatos de Office (`.docx`, `.xlsx`, `.pptx` son, por dentro, carpetas de archivos XML comprimidas), SVG, los feeds RSS/Atom, XSLT y buena parte de la integración de sistemas empresariales.

3.3. Clasificación de los lenguajes de marcas

Según el tipo de marca:

Tipo	Qué hace	Ejemplos
De presentación	Indica el formato o tipografía del texto, sin especificar su estructura	RTF, TeX, Markdown
Descriptivo / estructural / semántico	Indica las partes en que se estructura el documento, sin decir cómo debe representarse	XML, JSON

Tipo	Qué hace	Ejemplos
Híbrido	Contiene marcas de los dos tipos anteriores	HTML

Según su funcionalidad:

💡 Cada uno de los siguientes es un **lenguaje de marcas** (el formato), no un programa — el programa que lo interpreta es su *user-agent* correspondiente, como vimos en el punto 1.

- **Documentación electrónica:** Wikitexto, DocBook...
- **Tecnologías de Internet:** páginas web (HTML), formularios, mensajería instantánea — por ejemplo, XMPP/Jabber, un protocolo de mensajería basado enteramente en XML...
- **De propósito específico:** fórmulas matemáticas (**MathML**), síntesis de voz (**SSML** — el mismo que vimos en el punto 1 como ejemplo de user-agent), partituras musicales (**MusicXML**)...

🔧 **Ejercicio 3 — Diseña tu propio SGML** Elige un tema (por ejemplo: continentes, países, regiones, ciudades) y define:

- un **vocabulario** (los nombres de tus etiquetas),
- unas **reglas** (qué puede contener cada etiqueta y en qué orden),
- una **instancia** con datos reales siguiendo esas reglas.

Después, identifica en tu propio diseño cuál sería la **declaración**, cuál el **DTD** y cuál la **instancia** — las tres partes de un documento SGML vistas en el punto 3.1.

🔧 **Ejercicio 4 — Tu primera nota en Markdown** Escribe un resumen de los puntos 1 a 3 de esta unidad (8-12 líneas) en un archivo `.md`, usando al menos: dos niveles de encabezado (`#` y `##`), texto en **negrita** y *cursiva*, una lista, un bloque de código y una tabla de 2 columnas. Ábrelo con un visor de Markdown (por ejemplo, la vista previa de VS Code) y compara el texto plano con el resultado renderizado. Por último, según la clasificación del punto 3.3, ¿Markdown es una marca de presentación, estructural o híbrida? Razónalo.

4. Características de los lenguajes de marcas

- **Texto plano:** compuestos únicamente por caracteres de texto, codificables con distintos juegos de caracteres (ASCII, UTF-8...). Esto importa especialmente para nosotros: UTF-8 es lo que permite que un documento en valenciano o castellano con tildes, ces trencades o eñes se vea igual en cualquier sistema — y también soporta alfabetos no latinos (chino, árabe) o emojis. Interpretables con cualquier editor y, por tanto, independientes del sistema operativo.
- **Integración:** las marcas van incrustadas en el propio texto del documento (`<h2>Contenido</h2>`), en un único archivo — no en un archivo de contenido más un archivo aparte de instrucciones de formato sincronizado por posiciones, como hacían formatos más antiguos.
- **Independencia:** el mismo documento puede interpretarse de formas distintas según el dispositivo final (ver punto 2, donde lo desarrollamos con el ejemplo de HTML+CSS).
- **Especialización:** no son solo para la web — existen usos muy diversos: matemáticas (MathML), música (MusicXML), química (**CML**, *Chemical Markup Language*, para representar moléculas y reacciones), síntesis de voz (SSML)...
- **Flexibilidad:** se combinan fácilmente con otros lenguajes (ver punto 2, ejemplo de HTML + JavaScript/PHP).

💡 Esta lista es orientativa, no cerrada: son rasgos habituales, pero no una ley universal — por ejemplo, formatos como **EXI** codifican XML en binario, perdiendo la propiedad de "texto plano". Algunos LM tienen además propiedades extra: como veréis en el punto 5, XML es también **validable** y **extensible**, dos características que ni HTML ni Markdown tienen igual de desarrolladas.

🔧 **Ejercicio 5 — Características, a examen** Para cada uno de estos formatos, indica cuáles de las 5 características de este punto (texto plano, integración, independencia, especialización, flexibilidad) cumple y cuáles no, razonando tu respuesta:

- Un archivo **.txt** con la letra de una canción.
- Un documento HTML.
- Un archivo **.docx**.
- Una fotografía **.jpg**.

5. Ámbitos de aplicación y XML como lenguaje de propósito general (CA 1d, 1e)

Cuantos más dispositivos y sistemas distintos existen, más falta hace un estándar común para intercambiar información. Esto es lo que resuelve la **estandarización**: el proceso de fijar normas para que elementos contruidos de forma independiente (por ejemplo, dos aplicaciones de empresas distintas) funcionen bien juntos.

Las organizaciones que fijan estos estándares son principalmente el **W3C** (*World Wide Web Consortium*), **ISO** y la comunidad **Open Source**.

XML (*eXtensible Markup Language*) surge como el lenguaje de marcas de **propósito general**: un subconjunto de SGML pensado para ser más sencillo, con sintaxis más estricta, y que sirve para *cualquier* tipo de información estructurada, no solo para la web. Sus características principales son:

- Es un **metalenguaje**: permite crear infinitas etiquetas propias, adaptadas a cada necesidad — de hecho es justo lo que hicimos en el punto 3.1 al inventar las etiquetas **<daw>**, **<modulo>**, **<titulo>** para nuestro propio vocabulario. A diferencia de HTML, cuyo conjunto de etiquetas es fijo y lo define el WHATWG, en XML las etiquetas las decides tú según lo que necesites representar.
- Es **estructurado**: organiza los datos como un árbol jerárquico (lo veremos en el punto 6) sin pensar en cómo se presentarán — la misma separación de contenido y presentación que vimos en el punto 2.
- Es **validable**: se puede comprobar automáticamente, antes de procesarlo, si un documento cumple una estructura definida (un DTD o un XML Schema) — útil, por ejemplo, para rechazar de entrada un archivo mal formado que llega de otro sistema, sin tener que ejecutar código para descubrir el error.
- **No está limitado a la web**: se usa en cualquier tipo de aplicación — archivos de configuración, los formatos de Office por dentro (**.docx**, **.xlsx**), recursos de apps Android, intercambio de datos entre sistemas empresariales...

Algunos lenguajes contruidos a partir de XML:

Lenguaje	Uso
SVG	Gráficos vectoriales 2D
MathML	Fórmulas matemáticas

Lenguaje	Uso
SMIL	Información multimedia
SSML	Síntesis de voz

 **SVG, presente en tu día a día sin que lo notes:** cada icono, logo o ilustración vectorial que ves en la web suele ser SVG — incluido el logo `</>` de la portada de esta misma unidad, hecho exactamente así. Al ser XML, un navegador puede insertarlo directamente dentro del HTML (como en la portada) y modificarlo con CSS o JavaScript igual que cualquier otro elemento de la página — algo que una imagen de píxeles (`.png`, `.jpg`) no permite. Además es independiente de la resolución: el mismo archivo se ve nítido tanto en un icono pequeño como ampliado a pantalla completa, porque no está hecho de píxeles sino de fórmulas geométricas.

Icono de la portada:

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 400 400" width="180"
height="180">
  <circle cx="200" cy="200" r="170" fill="#F0FDFA"/>
  <circle cx="200" cy="200" r="170" fill="none" stroke="#99F6E4" stroke-
width="2"/>
  <text x="200" y="248" font-family="'Courier New', Courier, monospace" font-
size="150" font-weight="700" fill="#0F766E" text-anchor="middle">&lt;/&gt;</text>
</svg>
```

`viewBox="0 0 400 400"` define un lienzo de 400×400 unidades; `width/height="180"` es el tamaño final en pantalla. Las dos etiquetas `<circle>` dibujan el mismo círculo dos veces: una rellena de un verde muy suave (`fill="#F0FDFA"`) y otra solo con borde (`fill="none"`, `stroke="#99F6E4"`), para conseguir el efecto de círculo con contorno. `<text>` coloca el símbolo `</>` centrado (`text-anchor="middle"`), en Courier New y en verde oscuro (`#0F766E`) — fíjate que `<` y `>` van escritos como `<` y `>`, porque si se pusieran literalmente el navegador los confundiría con el inicio de otra etiqueta.

 **Ejercicio 6 — Tu propio icono SVG** Parte del código SVG de arriba y modifícalo para crear tu propio icono: cambia el texto, los colores (`fill`, `stroke`) o añade una segunda forma (por ejemplo un `<rect>` o un segundo `<circle>`). Guárdalo como `.svg` y ábrelo directamente con el navegador para ver el resultado. Después, responde: siendo una imagen, ¿por qué se puede editar con un editor de texto normal?

6. XML: estructura y sintaxis (CA 1f, 1g)

Un documento XML se organiza como un **árbol**: un único **elemento raíz** que contiene otros elementos anidados dentro.

```
<?xml version="1.0" encoding="UTF-8"?>
<nota>
  <de>Pablo</de>
  <para>Eric</para>
  <encabezado>Recordatorio</encabezado>
```

```
< cuerpo>Pablo, si no estudias, la IA no va a hacer que apruebes solo!</ cuerpo>  
</ nota>
```

Elementos de la sintaxis:

- **Declaración XML** (opcional pero recomendada): `<?xml version="1.0" encoding="UTF-8"?>` — indica la versión de XML y la codificación de caracteres usada. Casi siempre verás esta misma declaración; solo cambiaría si usaras otra codificación, por ejemplo `<?xml version="1.0" encoding="ISO-8859-1"?>` (una codificación más antigua, hoy en desuso frente a UTF-8).
- **Elemento**: la unidad básica, formada por una etiqueta de apertura y una de cierre. Ejemplos: `<de>Pablo</de>`, `<ciudad>Algemés</ciudad>`, `<precio>19.99</precio>` — el contenido puede ser texto, un número, o incluso otros elementos anidados.
- **Elemento raíz**: el elemento que engloba a todos los demás. Solo puede haber uno por documento. Aquí es `<nota>`; en el ejemplo del punto 3.1 era `<daw>`; en un archivo de configuración podría ser `<configuracion>` o `<settings>`.
- **Anidamiento**: los elementos pueden contener otros elementos dentro, formando una jerarquía (árbol). Por ejemplo:

```
< alumno>  
  < nombre>Claudia</ nombre>  
  < notas>  
    < nota>7</ nota>  
    < nota>8.5</ nota>  
  </ notas>  
</ alumno>
```

Aquí `<notas>` contiene varios `<nota>`, que a su vez están dentro de `<alumno>` — tres niveles de anidamiento.

- **Atributos**: información adicional dentro de la propia etiqueta de apertura, en pares `nombre="valor"`. Un elemento puede tener varios atributos a la vez:

```
< persona edad="20" ciudad="Valencia">Claudia</ persona>  
< libro isbn="978-84-376-0494-7" idioma="es">El Quijote</ libro>
```

- **Elemento vacío**: un elemento sin contenido se puede escribir con autocierre. Ejemplos: `<linea/>` en vez de `<linea></linea>`, o `<separador tipo="doble"/>` — un elemento vacío también puede llevar atributos.

 **Sigue vigente:** aunque hoy JSON gane en las APIs web, esta misma estructura de árbol es la que usan a diario los archivos de Office, los SVG y los feeds RSS.

 **Ejercicio 7 — De tabla a XML** Convierte esta tabla en un documento XML, usando `<discografia>` como elemento raíz, un elemento `<disco>` por fila, un atributo para el año, y un elemento vacío `<agotado/>` en los discos marcados como agotados. No olvides la declaración XML al principio.

Título	Año	Agotado
Álbum A	2019	No
Álbum B	2021	Sí
Álbum C	2023	No

7. Documentos XML bien formados (CA 1h)

Se dice que un documento XML está **bien formado** cuando cumple las reglas sintácticas básicas del lenguaje. Si no las cumple, ningún procesador XML podrá interpretarlo — dará error directamente, a diferencia de HTML, que muchos navegadores "perdonan" aunque tenga errores.

Reglas principales — con qué pasa si no se cumplen:

1. **Debe existir un único elemento raíz** que contenga a todos los demás.

✗ Mal:

```
<titulo>Curso</titulo>
<modulo>LMSGI</modulo>
```

Sin un elemento raíz común, no hay un único árbol que contenga todo el documento. El procesador ni siquiera llega a interpretar el contenido: da error de "documento sin elemento raíz" antes de leer nada más.

2. **Toda etiqueta que se abre debe cerrarse**: `<titulo>Texto</titulo>`, nunca dejarla abierta.

✗ Mal:

```
<titulo>Curso
```

Al llegar al final del documento sin encontrar `</titulo>`, el procesador lanza un error de "etiqueta sin cerrar" (*unclosed tag*).

3. **El anidamiento debe respetarse**: no se pueden cruzar etiquetas. `<a><b></a></b>` está mal; `<a><b></b></a>` está bien.

✗ Mal: `<a><b></a></b>` El procesador espera `</b>` (la última etiqueta abierta) y se encuentra `</a>`: error de "etiqueta de cierre inesperada" (*unexpected closing tag*).

4. **Los valores de los atributos siempre van entre comillas**: `edad="25"`, no `edad=25`.

✗ Mal:

```
<persona edad=20 ciudad=Valencia>Claudia</persona>
```

Sin comillas, el procesador no sabe dónde termina el valor de `edad` y dónde empieza el siguiente atributo: error de sintaxis al analizar los atributos.

5. **XML distingue mayúsculas de minúsculas:** `<Titulo>` y `<titulo>` son etiquetas distintas, y una no cierra a la otra.

✗ Mal: `<Titulo>Curso</titulo>` Para XML, `<Titulo>` y `</titulo>` son dos etiquetas diferentes, así que la de apertura nunca queda cerrada: el mismo error de "etiqueta sin cerrar" que en la regla 2.

6. Los elementos vacíos deben cerrarse con `/>` o con su etiqueta de cierre: `<linea/>` o `<linea></linea>`.

✗ Mal: `<linea>` Al no llevar ni `/>` ni una etiqueta de cierre `</linea>`, el procesador la trata como una etiqueta abierta que nunca se cierra: mismo error que en la regla 2.

En los seis casos el resultado es el mismo: el procesador XML se detiene con un error y no genera ningún documento, sin intentar "adivinar" qué querías decir — justo lo contrario de lo que hace HTML.

 **El intento histórico de aplicar esto a HTML:** a finales de los 90 se creó **XHTML**, una versión de HTML reescrita para obligar a cumplir estas mismas reglas de buen formato (cerrar siempre `<br/>` o `<img/>`, atributos entre comillas, minúsculas obligatorias). La idea era que las páginas web se pudieran procesar con las mismas herramientas que XML. En la práctica, **XHTML ha quedado en un segundo plano**: HTML5 absorbió parte de esa disciplina sin exigir la sintaxis estricta, y hoy XHTML apenas se usa en desarrollo nuevo. Aun así, entender el buen formato de XML te ayuda a escribir mejor HTML.

Ejercicio 8 — Bien formado, de la teoría a la práctica

Parte A: Escribe un documento XML bien formado que describa 3 alumnos de tu clase (nombre, edad, módulo favorito), aplicando las 6 reglas de este punto.

Parte B: A partir de tu propio documento de la Parte A, crea una copia "estropeada" introduciendo 3 o 4 errores deliberados de buen formato (por ejemplo: cambia una etiqueta a mayúsculas, elimina un cierre, cruza el anidamiento, quita las comillas de un atributo). Intercámbiala con un compañero, sin decirle qué has cambiado, y que identifique a qué regla corresponde cada error.

8. Espacios de nombres en XML (CA 1i)

Como en XML cualquiera puede inventarse sus propias etiquetas, es fácil que dos vocabularios que quieres combinar usen la **misma etiqueta con significados distintos**, o que uno de ellos necesite añadir información que el otro no contempla. Por ejemplo, un feed RSS (ya visto en el punto 3) solo define etiquetas genéricas (`<title>`, `<description>`...), pero las apps de podcasts necesitan datos extra que el RSS estándar no tiene: duración del episodio, autor, categoría, portada...

Los **espacios de nombres** (*namespaces*) resuelven este choque, asociando cada conjunto de etiquetas a una URI que las identifica de forma única. Así, un feed puede mezclar el vocabulario estándar de RSS con el

vocabulario propio de podcasts (definido originalmente por Apple, con prefijo **itunes**, y adoptado también por Spotify y otros) sin que haya ambigüedad:

```
<rss version="2.0" xmlns:itunes="http://www.itunes.com/dtds/podcast-1.0.dtd">
  <channel>
    <title>Podcast de Informática</title>
    <itunes:author>Noel Marco</itunes:author>
    <item>
      <title>Episodio 1: Introducción a XML</title>
      <itunes:duration>32:15</itunes:duration>
    </item>
  </channel>
</rss>
```

- `xmlns:itunes="..."` define un **prefijo** (**itunes**) asociado a una URI concreta.
- Las etiquetas sin prefijo (`<title>`) son RSS estándar: cualquier lector de feeds las entiende.
- Las etiquetas `itunes:algo` son el vocabulario añadido: un lector de feeds genérico que no las conoce las ignora sin romperse; una app de podcasts sí las interpreta para mostrar duración, autor, portada...
- La URI no es un enlace que nadie visite ni tiene que "existir" como página — es solo un identificador único. Por convención se usa un dominio del creador del vocabulario, para evitar que dos personas elijan el mismo prefijo por casualidad.



Sigue siendo necesario hoy: los espacios de nombres se usan continuamente en la práctica.

- Cuando insertas un `<svg>` dentro de una página HTML5, el navegador activa automáticamente el namespace de SVG en cuanto encuentra esa etiqueta (una regla especial de "contenido foráneo" del estándar HTML5) — no escribes el `xmlns` a mano, pero el mecanismo que lo hace posible es este.
- En feeds como el de arriba, para combinar el vocabulario estándar con el de podcasts, imágenes, geolocalización, etc. sin que las etiquetas choquen.
- Los propios archivos de Office (`.docx`, `.xlsx`, `.pptx`) son, por dentro, XML con varios espacios de nombres combinados (uno para el texto o las celdas, otro para estilos, otro para relaciones entre archivos internos del paquete) — la misma idea que ves aquí, a mayor escala.

 **Ejercicio 9 — Amplía el feed de podcast** Parte del ejemplo de feed de este punto y añade un segundo espacio de nombres inventado por ti (por ejemplo **miapp**, con una URI a tu elección) con al menos dos etiquetas propias (por ejemplo `<miapp:valoracion>` o `<miapp:transcripcion>`). Declara el `xmlns` correspondiente y usa el prefijo correctamente. Después, explica qué pasaría si, en lugar de un prefijo distinto, hubieras llamado a tu etiqueta simplemente `<title>`.

Resumen de la unidad

1. ¿Qué es un lenguaje de marcas? Un sistema de marcas insertadas en el propio texto (entre `<` y `>`) que un *user-agent* interpreta — el mismo documento puede dar resultados distintos según quién lo procese (navegador, editor, lector de pantalla, sintetizador de voz...). No es un lenguaje de programación.

2. Ventajas de los lenguajes de marcas Texto plano (editable sin software específico, comparable línea a línea con Git), independencia del dispositivo (separación entre contenido y presentación), facilidad de intercambio entre sistemas distintos, y flexibilidad para combinarse con otros lenguajes.

3. Origen y clasificación GML → SGML (metalenguaje, estandarizado por ISO en 1986) → de ahí nacen XML (subconjunto simplificado) y HTML (aplicación de SGML interpretada de forma permisiva). Otros lenguajes de marcas: LaTeX, RTF, PostScript, JSON, Markdown. Se clasifican por tipo de marca (presentación / estructural / híbrida) y por funcionalidad.

4. Características de los lenguajes de marcas Texto plano, integración, independencia, especialización y flexibilidad — rasgos habituales, no una ley cerrada (hay excepciones, como el binario EXI).

5. Ámbitos de aplicación y XML de propósito general XML es un metalenguaje, estructurado en árbol, validable y no limitado a la web. Lenguajes construidos sobre XML: SVG, MathML, SMIL, SSML.

6. XML: estructura y sintaxis Declaración XML, elemento raíz único, anidamiento, atributos y elementos vacíos — la base sintáctica para escribir cualquier documento XML.

7. Documentos XML bien formados Seis reglas sintácticas obligatorias (raíz única, cierre de etiquetas, anidamiento correcto, atributos entre comillas, sensibilidad a mayúsculas, elementos vacíos cerrados). Si no se cumplen, el procesador da error directamente — a diferencia de HTML.

8. Espacios de nombres en XML Resuelven los choques de etiquetas entre vocabularios distintos, asociando cada uno a una URI única mediante prefijos (`xmlns:prefijo="URI"`) — imprescindibles al combinar formatos (SVG en HTML, feeds, archivos de Office).



Para saber más (opcional, no evaluable)

- W3Schools: <https://www.w3schools.com/>
- HTML Living Standard (WHATWG): <https://html.spec.whatwg.org/>